

11. Seguridad

PDF Reader v0.3.0 | Análisis 2026-09-10 | Commit 265b122

Contenido

- Modelo de amenazas
- Controles comprobados
- Datos personales y privacidad
- Validación y sanitización
- Riesgos
- CORS, CSRF e inyección
- Secretos y respuesta
- Recomendaciones prioritarias

Modelo de amenazas

Los PDF, nombres y rutas son entradas no confiables. Los activos principales son el contenido privado del documento, el almacenamiento local, la integridad del original, la clave de firma Android y el entorno del usuario. No hay identidad, autenticación, roles ni sesiones porque no existe servicio multiusuario.

Controles comprobados

Superficie	Control
Alcance	no hay funciones de guardar/editar/anotar; verificador bloquea marcadores prohibidos
Electron	contextIsolation:true, nodeIntegration:false, sandbox:true
IPC	preload expone una lista cerrada; lectura valida extensión .pdf
Navegación	nuevas ventanas denegadas desde Electron
Android	sin INTERNET ni permisos generales de almacenamiento
Android intents	acceso por URI concedida y FileProvider no exportado
Android memoria	apertura externa limitada a 128 MiB
Web	mixed content desactivado en Capacitor
Búsqueda	contenido y consulta se escapan antes de innerHTML
Persistencia	borrado por entrada o completo mediante UI
CI	auditoría de permisos APK y dependencias de producción
Supply chain	versiones fijadas, lockfile congelado, Dependabot
Release	zipalign, apksigner, verificación y SHA-256

Datos personales y privacidad

El contenido, nombre y progreso del PDF pueden ser sensibles. Permanecen en memoria/IndexedDB local salvo que la persona comparta voluntariamente el nombre/progreso. No existe cifrado de aplicación; se depende del aislamiento del perfil y del sistema operativo. GitHub Pages puede registrar accesos al sitio, pero el código no sube archivos ni integra telemetría.

Validación y sanitización

La selección Web revisa extensión o MIME; Electron exige extensión; Android confía en intent/MIME y parser PDF.js. Esto no valida la firma mágica antes de cargar. PDF.js trata el formato y devuelve error. Los snippets usan `escapeHtml`; el resto de nombres se escribe con `textContent`.

Riesgos

- PDF malicioso contra una vulnerabilidad de PDF.js/WebView/Electron: mantener dependencias actualizadas y revisar advisories.
- DoS de memoria: Web/Windows no tienen límite explícito y Android Base64 multiplica copias.
- IPC `openPath`: cualquier código que comprometiera el renderer podría solicitar lectura de una ruta `.pdf`; el `sandbox/preload` reducen, no eliminan, este impacto.
- Persistencia local sin cifrado ni expiración temporal.
- `allowBackup=true` puede permitir que Android incluya datos privados según política/dispositivo; requiere decisión explícita.
- Ausencia de CSP visible en `src/index.html`; una CSP estricta reduciría impacto de inyección.
- Windows sin `Authenticode`: SmartScreen y menor garantía de procedencia.
- Las GitHub Actions se referencian por tags mayores (`@vN`) mutables, no por SHA inmutable; una cuenta upstream comprometida podría alterar el código ejecutado por CI.

CORS, CSRF e inyección

No hay endpoints ni cookies de sesión, por lo que CSRF no aplica. CORS no se configura porque no se consume una API del producto. Sí aplican XSS/HTML injection: el único `innerHTML` con datos del PDF pasa por escape; `defaultText.innerHTML` usa texto constante. No se encontraron SQL, `eval` de contenido ni comandos shell formados desde datos del PDF. Android ejecuta un JavaScript constante al anunciar intent nuevo.

Secretos y respuesta

La búsqueda estática no identifica secretos reales versionados; solo nombres de GitHub Secrets. Las vulnerabilidades deben informarse mediante Security Advisories sin adjuntar PDF privados. No hay logging/auditoría de runtime; los workflows conservan logs de build.

Recomendaciones prioritarias

- 1 decidir y probar `android:allowBackup` respecto del historial;
- 2 imponer límites configurables de tamaño en todas las plataformas;
- 3 añadir CSP compatible con PDF.js;
- 4 incorporar pruebas E2E de IPC/intents y fuzz/regresión con corpus no sensible;
- 5 firmar Windows con `Authenticode` cuando exista infraestructura de certificados.
- 6 fijar Actions a SHA completos y mantener comentarios con su versión legible.